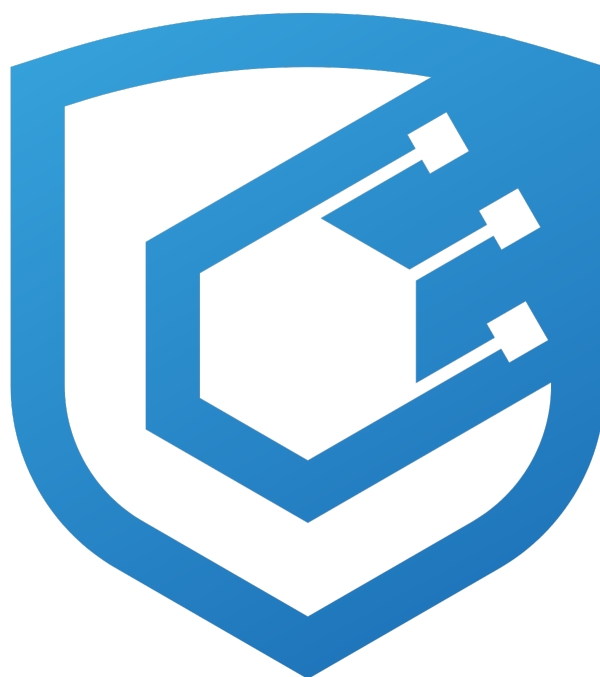




Atum TRON Contracts Audit Report

Prepared by [Cyfrin](#)

Version 2.1

Lead Auditors

[Dacian](#)

[Al-qaqa](#)

August 27, 2025

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	3
6.1	TRON Token Standard Compatibility	3
6.2	Centralization and Trust Assumptions	3
6.2.1	Token Issuer Risks	3
6.2.2	Protocol Admin Powers	3
6.2.3	Security Boundaries	3
6.2.4	Trust Model	4
7	Findings	6
7.1	Critical Risk	6
7.1.1	USDT tokens on TRON used as deposits will be permanently stuck in Escrow contract as Escrow::release, refund revert due to critical bug inside USDT's transfer function	6
7.2	Low Risk	7
7.2.1	Fulfillment::requestHash should reference depositId generated inside Escrow::deposit	7
7.2.2	Smart contract wallets using Approved Hashes are limited to one active deposit at a time . .	7
7.3	Informational	8
7.3.1	Use TRON isContract to determine whether an address is a contract	8
7.3.2	TIP-712 requires addresses to be cast to uint160 but address in TRON Solidity doesn't store the prefix so casting is unnecessary	8
7.3.3	Incorrect description of depositId generation in IEscrow	8

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Atum is a cross-chain stablecoin payments protocol that leverages an off-chain Request for Quote (RFQ) mechanism to facilitate seamless token transfers across different blockchains and stablecoin types. The protocol's innovative design abstracts away blockchain complexity from the end users (Originators), who simply need to sign a Permit2 message to initiate a payment request - they never need to interact directly with smart contracts or manage cross-chain messaging.

When an Originator needs to pay in stablecoins across chains (e.g., deposit USDC on Ethereum but pay using DAI on Arbitrum), they publish an RFQ containing their signed Permit2 authorization, which multiple liquidity providers (Settlers) can bid on competitively. The winning Settler, designated by a trusted Reserver, locks the source funds in the Escrow contract and fulfills the swap on the destination chain, after which a trusted Releaser verifies the fulfillment and authorizes the release of escrowed funds to the Settler.

This architecture pushes all technical complexity - including liquidity management, cross-chain execution, and blockchain interactions - to sophisticated Settlers who compete to provide the best rates, while Originators enjoy a simple, signature-based experience. The protocol employs a trust-minimized design where the Escrow contract ensures atomic settlement through cryptographic signatures and role separation, with the Reserver controlling settler assignment and the Releaser validating cross-chain fulfillment, making it particularly suitable for applications like cross-border payments, remittances, and chain-agnostic stablecoin transfers where user simplicity is paramount.

5 Audit Scope

Cyfrin has previously audited Atum's EVM Contracts. The scope of this audit was Atum's TRON contracts and the fork of the Permit2 library for TRON deployment:

```
tvm-contracts/contracts/Escrow.sol
tvm-contracts/contracts/FulfillmentProxy.sol
tvm-contracts/contracts/TIP712.sol
permit2-tron/tron.diff
```

In addition to the `Permit2` diff, we also read the remaining unchanged `Permit2` code looking for any places where necessary changes may have been omitted.

6 Executive Summary

Over the course of 3 days, the Cyfrin team conducted an audit on the [Atum TRON Contracts](#) smart contracts provided by [Atum Labs](#). In this period, a total of 6 issues were found.

The findings consist of 1 Critical and 2 Lows with the remainder being informational and gas optimizations. The single Critical finding resulted in `USDT` tokens being permanently stuck inside the `Escrow` contract due to a bug in the `transfer` function in `TRON USDT` which always returns `false` even when a transfer is successful.

6.1 TRON Token Standard Compatibility

The `Escrow` contract exclusively supports `TRC-20` tokens (TRON's equivalent of `ERC-20`) and does not support `TRC-10` tokens. This design decision aligns with the modern TRON ecosystem where `TRC-10` tokens have become largely obsolete. `TRC-10` tokens are native tokens built into the TRON protocol layer that:

- are identified by numeric token IDs rather than contract addresses
- lack smart contract functionality and cannot implement the `approve/transferFrom` pattern that `Permit2` relies upon for its signature-based transfer mechanism

This limitation is intentional and does not represent a security risk or functional deficiency. All major stablecoins on TRON (`USDT`, `USDC`, `TUSD`) and actively traded tokens are implemented as `TRC-20` contracts. The [TIP-712](#) specification includes encoding rules for `TRC-10` tokens (as `trcToken` encoded as `uint256`), but these rules are not applicable to this protocol since all token addresses in the `Escrow` and `Permit2` components are `TRC-20` contract addresses that are correctly encoded as `uint160` per the [TIP-712](#) standard.

Users attempting to escrow `TRC-10` tokens would need to first convert them to `TRC-20` equivalents through official migration contracts if available, though most significant `TRC-10` tokens like BitTorrent Token (`BTT`) have already completed migrations to `TRC-20` standards.

6.2 Centralization and Trust Assumptions

The protocol is designed to work with centralized stablecoins that implement blacklist functionality. This introduces several trust assumptions and centralization vectors:

6.2.1 Token Issuer Risks

Both protocol contract addresses and participant addresses (`Depositor`, `Reserver`, `Releaser`, `Settler`) can be blacklisted by token issuers, potentially freezing tokens indefinitely within the protocol. This creates an external dependency where the protocol's functionality can be disrupted by entities outside the protocol's control.

6.2.2 Protocol Admin Powers

Protocol admins (contract owner) have limited but important powers:

- **Token Management:** Enable and disable supported tokens via the allowlist. Note that disabling a token does not affect existing active deposits, allowing them to complete or be refunded normally
- **Emergency Controls:** Pause and unpause the entire `Escrow` contract, halting all user operations

6.2.3 Security Boundaries

Importantly, protocol admins **cannot**:

- Withdraw or "rescue" tokens from the `Escrow` contract (no backdoor functions exist)

- Upgrade the contract implementation (contract is immutable)
- Alter or interfere with active deposits
- Override signature requirements or bypass the cryptographic validation

6.2.4 Trust Model

While privileged roles (Reserver, Releaser, Settler) exist within the protocol, these are effectively controlled by users through cryptographic commitments:

- The Depositor specifies both Reserver and Releaser addresses, which are cryptographically committed via the DepositWitness signature
- The Reserver specifies the Settler, but since the Reserver was chosen by the Depositor, this maintains the chain of trust
- All role transitions require valid TIP-712 signatures, preventing unauthorized changes

This design creates a hybrid trust model: trustless between protocol participants (enforced by cryptography), but requiring trust in token issuers (who can blacklist) and minimal trust in protocol admins (who can pause operations but not steal funds).

Summary

Project Name	Atum TRON Contracts
Repository	tvm-contracts
Commit	e6d5e9f79552...
Repository 2	permit2-tron
Commit	a31ae2274278...
Audit Timeline	Aug 18th - Aug 20th, 2025
Methods	Manual Review

Issues Found

Critical Risk	1
High Risk	0
Medium Risk	0
Low Risk	2
Informational	3
Gas Optimizations	0
Total Issues	6

Summary of Findings

[C-1] USDT tokens on TRON used as deposits will be permanently stuck in Escrow contract as <code>Escrow::release</code> , refund revert due to critical bug inside USDT's transfer function	Resolved
[L-1] <code>Fulfillment::requestHash</code> should reference <code>depositId</code> generated inside <code>Escrow::deposit</code>	Resolved
[L-2] Smart contract wallets using Approved Hashes are limited to one active deposit at a time	Acknowledged
[I-1] Use TRON <code>isContract</code> to determine whether an address is a contract	Acknowledged
[I-2] TIP-712 requires addresses to be cast to <code>uint160</code> but address in TRON Solidity doesn't store the prefix so casting is unnecessary	Resolved
[I-3] Incorrect description of <code>depositId</code> generation in <code>IEscrow</code>	Resolved

7 Findings

7.1 Critical Risk

7.1.1 USDT tokens on TRON used as deposits will be permanently stuck in Escrow contract as Escrow::release, refund revert due to critical bug inside USDT's transfer function

Description: The official TRON USDT token address is given as [TR7NHqjeKQxGTCi8q8ZY4pL8otSzgjlj6t](#).

The relevant inheritance heirarchy is TetherToken -> StandardTokenWithFees -> StandardToken -> BasicToken.

There appears to be one significant bug inside StandardTokenWithFees::transfer which does this:

```
function transfer(address _to, uint _value) public returns (bool) {
    uint fee = calcFee(_value);
    uint sendAmount = _value.sub(fee);

    super.transfer(_to, sendAmount);
    if (fee > 0) {
        super.transfer(owner, fee);
    }
}
```

The bug here is there is no return statement so this will return false even for successful transfers.

TetherToken::transfer will call this function if deprecated == false which is [currently the case](#). The transferFrom function is fine as that has the return, it is only transfer which is the problem.

Impact: In Atum's codebase Escrow::release, refund call safeTransfer which under-the-hood calls IERC20::transfer. When token == USDT even using safeTransfer this bad bug causes the successful transfer to revert due to StandardTokenWithFees::transfer returning false. This results in escrowed USDT tokens being permanently stuck inside the Escrow contract.

Recommended Mitigation: In Escrow::release, refund when token == USDT just call usdt.transfer and ignore the return value - this is safe as it will always revert if the transfer failed.

Atum: Fixed in commit [12fb91a](#).

Cyfrin: Verified.

7.2 Low Risk

7.2.1 Fulfillment::requestHash should reference depositId generated inside Escrow::deposit

Description: Currently Fulfillment::requestHash (emitted in the Fulfilled event on the destination chain) references DepositWitness::requestId.

However this is not very useful and potentially dangerous when reconciling fulfillments to requests off-chain, since the input DepositWitness::requestId:

- is never stored anywhere on-chain
- may not be unique as it is an arbitrary input supplied by the Originator

Recommended Mitigation: The Fulfilled event should instead reference the depositId which is created inside Escrow::deposit, is always unique and is also stored on-chain.

Atum: Fixed in commit [7049c6c](#) for tvn-contracts and [3dc4563](#) for evm-contracts.

Cyfrin: Verified.

7.2.2 Smart contract wallets using Approved Hashes are limited to one active deposit at a time

Description: Escrow::deposit generates the unique depositId based on the depositor's address and their signature:

```
depositId = keccak256(abi.encode(depositor, signature));
```

The problem in this is that the signature is always 65/64 length for EOA wallets, but it can be with different values in case of Smart Contract Wallets.

Some Smart Contract Wallets like Safe Wallet have a concept of Approved Hashes where the owners of the wallet can approve a given hash, then after signing the hash it is moved to Approved hashes mapping or it can be approved before that.

The problem is that in order to use the Approved Hashes feature you should pass signature of zero length, so as to let the verification go through Approved Hash: [SafeWallet::SignatureVerifierMuxer.sol#L154-L156](#)

```
function defaultIsValidSignature(ISafe safe, bytes32 _hash, bytes memory signature) internal view
↳ returns (bytes4 magic) {
    bytes memory messageData = EIP712.encodeMessageData(
        safe.domainSeparator(),
        SAFE_MSG_TYPEHASH,
        abi.encode(keccak256(abi.encode(_hash)))
    );
    bytes32 messageHash = keccak256(messageData);
>> if (signature.length == 0) {
        // approved hashes
>>     require(safe.signedMessages(messageHash) != 0, "Hash not approved");
    } else {
        // threshold signatures
        safe.checkSignatures(address(0), messageHash, signature);
    }
>> magic = ERC1271.isValidSignature.selector;
}
```

So in order for Smart Contract wallets relying on Approved Hashes, they will provide signature of zero length. In the case they have an active deposit, they will be prevented from making another deposit unless the old one gets completed (released/refunded), limited users to one active deposit.

Impact: Users of Smart Contract wallets using approved hashes are limited to one active deposit.

Recommended Mitigation: In Escrow::deposit also use permit.nonce when generating depositId.

Atum: Acknowledged; will be fixed in a future version.

7.3 Informational

7.3.1 Use TRON `isContract` to determine whether an address is a contract

Description: TRON has a special opcode `ISCONTRACT` added in [TIP-44](#) for determining whether an address is a contract or not.

Use this in the permit2 fork at <https://github.com/alexroan/permit2-tron/blob/main/contracts/libraries/SignatureVerification.sol#L26> doing something like:

```
-         if (claimedSigner.code.length == 0) {  
+         if (!claimedSigner.isContract) {
```

However the existing code appears to function correctly as well so there is no significant reason to change this. `SignatureChecker::isValidSignatureNow` used by the Escrow contract also uses the [same code length](#).

Atum: Acknowledged.

7.3.2 TIP-712 requires addresses to be cast to `uint160` but address in TRON Solidity doesn't store the prefix so casting is unnecessary

Description: [TIP-712](#) requires addresses used in TIP-712 to be cast to `uint160`:

- address: need to remove TRON unique prefix(0x41) and encoded as `uint160`

The encoding of a struct instance is `enc(value) || enc(value) || ... || enc(value)`, i.e. the concatenation of the encoded member values in the order that they appear in the type. Each encoded member value is exactly 32-byte long.

It's totally compatible with EIP-712.

The only difference between TRON address and Ethereum address is that TRON address starts with a byte prefix 0x41 and uses base58 encoding, so prefix needs to be removed when the address type is processed.

However address in TRON Solidity is already 20 bytes without any prefix, therefore casting address to `uint160` wouldn't "remove TRON unique prefix" - it wouldn't do anything except increase transaction costs.

Recommended Mitigation: * In `tvm-contracts`, the following `uint160(address)` casts can be safely removed:

```
TIP712.sol  
45:         return keccak256(abi.encode(typeHash, nameHash, versionHash, chainId,  
↳ uint160(address(this))));
```

- In `permit2-tron`, the following `uint160(address)` casts can be safely removed:

```
TIP712.sol  
38:         return keccak256(abi.encode(typeHash, nameHash, chainId, uint160(address(this))));  
  
libraries/PermitHash.sol  
40:         abi.encode(_PERMIT_SINGLE_TYPEHASH, permitHash, uint160(permitSingle.spender),  
↳ permitSingle.sigDeadline)  
54:         uint160(permitBatch.spender),  
64:         _PERMIT_TRANSFER_FROM_TYPEHASH, tokenPermissionsHash, uint160(msg.sender),  
↳ permit.nonce, permit.deadline  
81:         uint160(msg.sender),  
97:         abi.encode(typeHash, tokenPermissionsHash, uint160(msg.sender), permit.nonce,  
↳ permit.deadline, witness)  
120:         uint160(msg.sender),  
132:         _PERMIT_DETAILS_TYPEHASH, uint160(details.token), details.amount,  
↳ details.expiration, details.nonce  
143:         return keccak256(abi.encode(_TOKEN_PERMISSIONS_TYPEHASH, uint160(permitted.token),  
↳ permitted.amount));
```

The only time a uint160 cast makes sense is if an address is passed as external input using bytes or uint256 and it also contains the TRON prefix.

Atum: Fixed in commit [e0b9ce3](#) for permit2-tron and commit [0772f28](#) for tvn-contracts.

Cyfrin: Verified.

7.3.3 Incorrect description of depositId generation in IEscrow

Description: In IEscrow.sol file, it is mentioned that depositId is generated as the keccak256(signature), and this is incorrect. the depositId is generated using both depositor address and the signature

[IEscrow.sol#L207](#)

```
/// @notice Deposit tokens into the escrow using Permit2 signature
>> /// @dev The returned depositId is generated as keccak256(signature) and must be used for all
↳ subsequent operations
/// @param permit Permit2 permit structure containing token, amount, nonce, and deadline
/// @param depositor Address that owns the tokens and signed the permit
/// @param witness Contains requestId (user reference) and authorized reserver/releaser addresses
/// @param signature EIP-712 signature from depositor authorizing the transfer
/// @return depositId Unique identifier for this deposit (keccak256(signature)) - use for
↳ reserve/release/refund
function deposit( ... )
```

[Escrow.sol#L100-L102](#)

```
function deposit( ... ) external whenNotPaused returns (bytes32 depositId) {
    ...

    // Generate a deposit ID from the depositor and signature
    // This prevents griefing attacks where a malicious depositor can use the signature of a
    ↳ legitimate deposit
>> depositId = keccak256(abi.encode(depositor, signature));
    ...
}
```

Impact:

- Incorrect docs leading to error in description of how the contract works

Proof of Concept: **Recommended Mitigation:** correct the mistake by making it generated as keccak256(depositor, signature)

```
diff --git a/1-tvm-contracts/contracts/IEscrow.sol b/1-tvm-contracts/contracts/IEscrow.sol
index 914559a..9d21d99 100644
--- a/1-tvm-contracts/contracts/IEscrow.sol
+++ b/1-tvm-contracts/contracts/IEscrow.sol
@@ -204,7 +204,7 @@ interface IEscrow {
    // Core Functions

    /// @notice Deposit tokens into the escrow using Permit2 signature
-   /// @dev The returned depositId is generated as keccak256(signature) and must be used for all
↳ subsequent operations
+   /// @dev The returned depositId is generated as keccak256(depositor, signature) and must be used
↳ for all subsequent operations
    /// @param permit Permit2 permit structure containing token, amount, nonce, and deadline
    /// @param depositor Address that owns the tokens and signed the permit
    /// @param witness Contains requestId (user reference) and authorized reserver/releaser addresses
```

Atum: Fixed in commit [0772f28](#) for tvn-contracts and [f668872](#) for evm-contracts.

Cyfrin: Verified.